Task 1. Provision `music-streaming-server`

Use the EC2 configuration from Section B, changing the resource to:

```
None
resource "aws_instance" "music_server" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"

  tags = {
    Name = "music-streaming-server"
  }
}
```

This creates an EC2 instance with the requested name tag.

Task 2. Split the Terraform configuration into files

Terraform automatically loads all `.tf` files in the same directory. Create the following project structure:

```
None
terraform-music/
├── main.tf
├── ec2.tf
├── variables.tf
├── db.tf
├── outputs.tf
└── .gitignore
```

`main.tf`

```
None
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }
}
```

variables.tf

None

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
```

```
  default     = "t2.micro"
}

variable "instance_name" {
  description = "EC2 Name tag"
  type        = string
  default     = "music-streaming-server"
}

variable "db_password" {
  description = "RDS master password"
  type        = string
  sensitive   = true
}
```

`ec2.tf`

```
None
resource "aws_instance" "music_server" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = var.instance_type

  tags = {
    Name = var.instance_name
  }
}
```

Explanation: Input variables allow the region, instance type, and instance name to be changed without editing the resource itself.

Task 3. Create the RDS MySQL database

Add the following to `db.tf`. This is a demonstration configuration; it uses private database access and deliberately does not expose MySQL to the public internet.

None

```
resource "aws_db_subnet_group" "playlist" {
  name       = "playlist-db-subnets"
  subnet_ids = data.aws_subnets.default.ids

  tags = {
    Name = "playlist-db-subnets"
  }
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

resource "aws_security_group" "rds" {
  name        = "playlist-db-sg"
  description = "Allow MySQL only from the music server"
  vpc_id      = data.aws_vpc.default.id

  ingress {
    from_port       = 3306
    to_port         = 3306
    protocol        = "tcp"
    security_groups = [aws_security_group.app.id]
  }
}

resource "aws_security_group" "app" {
  name        = "playlist-app-sg"
  description = "Application security group"
```

```
  vpc_id      = data.aws_vpc.default.id
}

resource "aws_db_instance" "playlist" {
  identifier             = "playlist-db"
  engine                 = "mysql"
  instance_class         = "db.t3.micro"
  allocated_storage      = 20
  db_name                = "playlist"
  username               = "playlistadmin"
  password               = var.db_password
  db_subnet_group_name   = aws_db_subnet_group.playlist.name
  vpc_security_group_ids = [aws_security_group.rds.id]

  publicly_accessible    = false
  skip_final_snapshot    = true
  deletion_protection    = false
}
```

This example illustrates a private RDS database and a security group that accepts MySQL traffic only from the application security group. For an actual deployment, attach `aws_security_group.app` to the EC2 instance as well.

Note: A subnet group should use subnets in at least two Availability Zones. Check that your default VPC has suitable subnets across two zones before applying. The `skip_final_snapshot` and disabled deletion protection settings are for disposable lab use only; use backups and deletion protection for important data.

`outputs.tf`

```
None
output "ec2_public_ip" {
  description = "Public IPv4 address of the EC2 instance"
  value       = aws_instance.music_server.public_ip
}

output "rds_endpoint" {
```

```
  description = "RDS database endpoint"
  value       = aws_db_instance.playlist.endpoint
}
```

Initialize and validate:

```
None
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

Supply the database password securely, for example using the `TF_VAR_db_password` environment variable. Avoid saving it in source control or committing it in a `.tfvars` file.

Task 4. Add Git version control and a README

`.gitignore`

```
None
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
terraform.tfvars
*.pem
.env
```

Terraform state can contain sensitive values, so it must not be committed to Git. For team projects, use a secure remote backend with access control, encryption, and state locking.

Run:

```
None
git init
git add main.tf ec2.tf variables.tf db.tf outputs.tf .gitignore
```

```
git commit -m "Add Terraform music infrastructure"
```

Create `README.md`:

```
None

# Music Streaming Infrastructure

## Prerequisites
- Terraform
- AWS CLI
- An AWS account with appropriate permissions

## Deployment
1. Configure AWS credentials.
2. Review variables and region settings.
3. Run `terraform init`.
4. Run `terraform fmt` and `terraform validate`.
5. Run `terraform plan`.
6. Supply the database password securely.
7. Run `terraform apply`.
8. Review outputs using `terraform output`.

## Cleanup
Run `terraform destroy` after verifying that the resources
and any data stored in the database are no longer required.

## Security
- Do not commit credentials, private keys, or Terraform state.
- Keep the database private.
- Restrict network access using security groups.
```

Task 5. Three Terraform best practices

1. Version control: Store configuration in Git and review changes before merging. This helps developers track changes, review mistakes, and roll back faulty configurations.

2. Reusable modules and variables: Separate networking, compute, and database resources into reusable modules. This reduces duplication and makes environments more consistent.
3. Secure state and credentials: Use a protected remote backend, locking, encryption, and a secret manager. This reduces the risk of exposing credentials and prevents concurrent state changes from corrupting deployments.